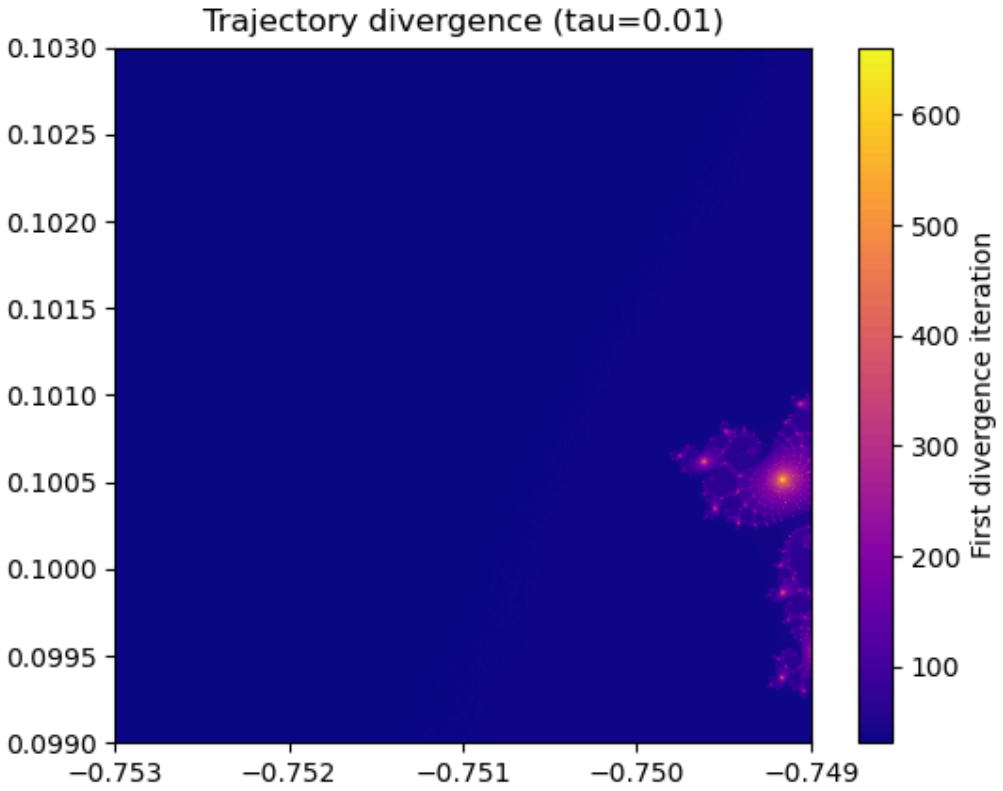
Lecture 8 Milestone 1: Mandelbrot Trajectory Divergence



What fraction of pixels diverge before max iter?
-> fraction = np.mean(diverge < MAX_ITER)
print(fraction)
The fraction is 1.0, meaning all pixels have diverged before the maximum iteration of 1000. The reason may be that the tau is very small, 0.01.

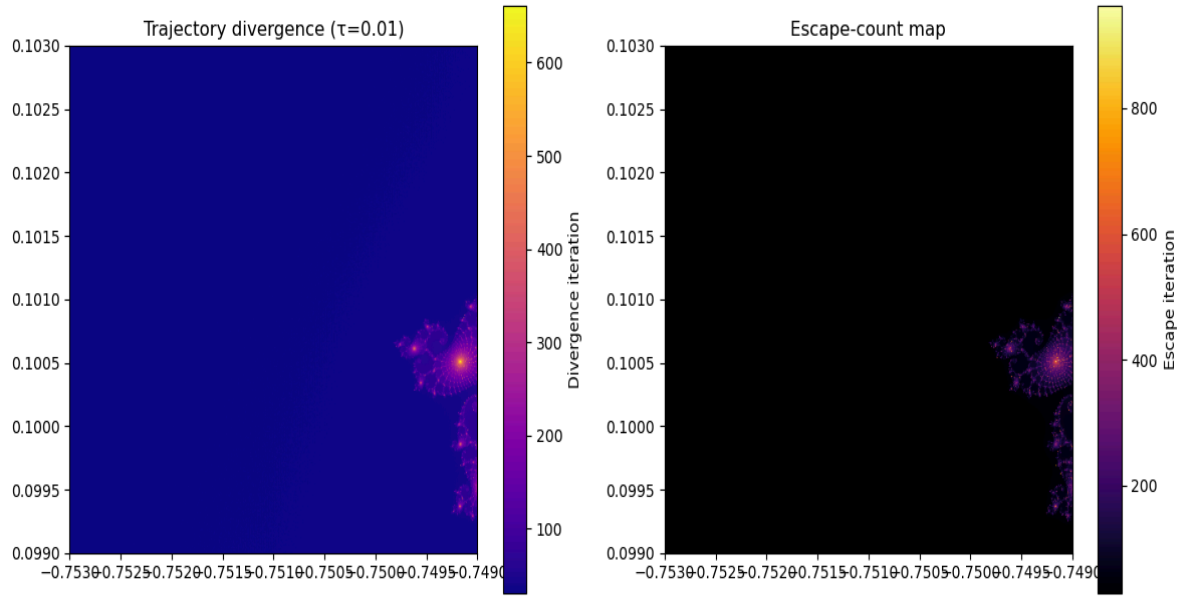
Where do trajectories diverge early? Compare visually to the escape-count map.
-> Looking at the above figure, it can be seen that almost all the parts of the figure are dark, which means those pixels diverged early. The edge pixels diverged earlier than the inside pixels.

Does early divergence correlate with high escape iteration counts?
->

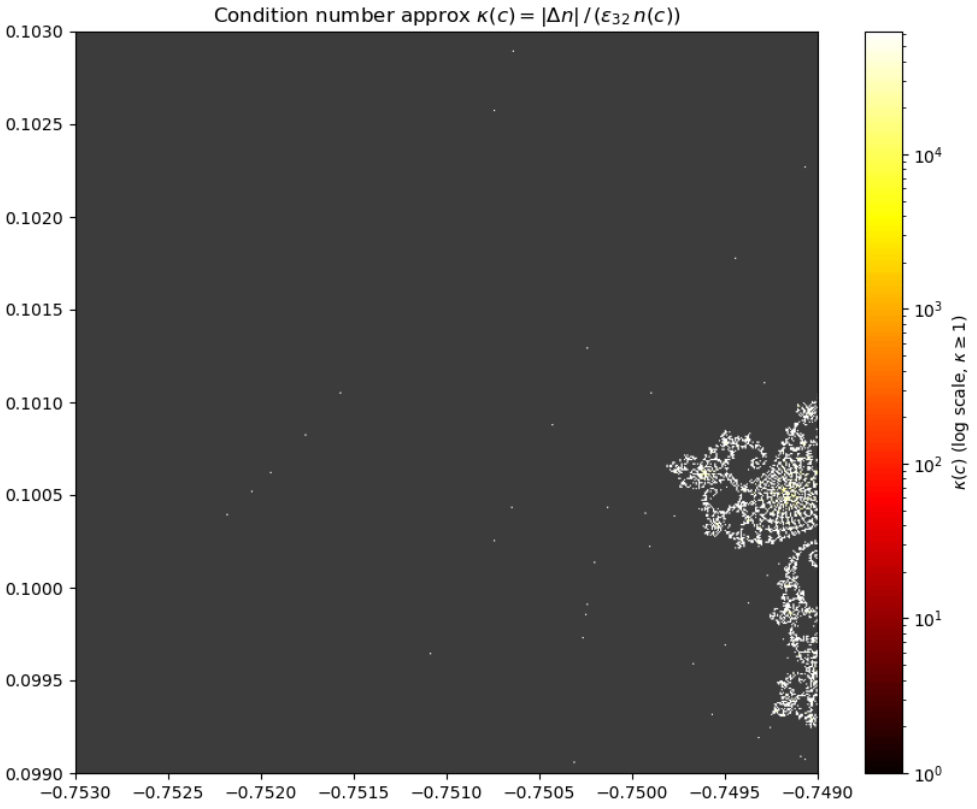
```
# Correlation check
mask = escape < MAX_ITER  # only escaped pixels
correlation = np.corrcoef(diverge[mask], escape[mask])[0, 1]
print(f"Correlation (escaped pixels only): {correlation:.4f}")
```

Output: Correlation (escaped pixels only): 0.9195

Yes, but inversely. Meaning pixels with high escape iterations diverge late. On the other hand, pixels with low escape iterations diverge fast. Pixels that are far from the boundary escape quickly, and their float32 and float64 trajectories drift apart faster.



Lecture 8 Milestone 2: Mandelbrot Sensitivity Map



Where is κ largest? Does it match the boundary in M1?
-> At the bottom right corner, k is the largest. Yes, it matches the boundary in M1. The same pixels are numerically unstable and ill-conditioned.

What is κ for interior pixels (n = max iter)?
-> The interior pixels appear gray (NaN) in the figure, which means they are well-conditioned (stable). Even a change (delta) in the input did not impact the output (delta_n = 0). It does not matter how many iterations we do; those pixels never escape.

Comparing the two figure forms: in milestone 1, the dark blue region shows early divergence (float32 and float64 diverge early), while in milestone 2, the dark grey region indicates well-conditioned behavior (not affected by small input changes). Bright regions in M1 indicate that, in those pixels, float32 lasted longer, and in M2, a bright color region indicates an ill-conditioned boundary. Both figures highlight the same structure, indicating that the region is where float32 precision loss and ill-conditioning occur.

Lecture 9 Milestone 1: Test Suite

```
test_mandelbrot.py::test_numpy_matches_naive PASSED                    [  9%]
test_mandelbrot.py::test_numba_matches_naive PASSED                    [ 18%]
test_mandelbrot.py::test_parallel_matches_naive PASSED                 [ 27%]
test_mandelbrot.py::test_pixel_inside PASSED                           [ 36%]
test_mandelbrot.py::test_pixel_outside PASSED                          [ 45%]
test_mandelbrot.py::test_chunk_values_non_negative PASSED              [ 54%]
test_mandelbrot.py::test_parallel_matches_serial PASSED                [ 63%]
test_mandelbrot.py::test_parallel_consistency[16] PASSED               [ 72%]
test_mandelbrot.py::test_parallel_consistency[32] PASSED               [ 81%]
test_mandelbrot.py::test_parallel_consistency[64] PASSED               [ 90%]
test_mandelbrot.py::test_dask PASSED                                   [100%]

==================================== 11 passed in 2.33s ====================================
```

Module	Statements	Missed	Coverage
benchmark.py	55	55	0%
mini_project_1/mandelbrot_naive.py	31	9	71%
mini_project_1/mandelbrot_naive_numba.py	53	41	23%
mini_project_1/mandelbrot_numpy.py	33	14	58%
mini_project_1/test_install.py	22	12	45%
mini_project_2/mandelbrot_parallel_chunk_lec5_m3.py	90	62	31%
test_mandelbrot.py	56	0	100%

Metric	Value
Total statements	340
Total missed	193
Overall coverage	43%

Lecture 9 Milestone 2: Docstrings and Type Hints

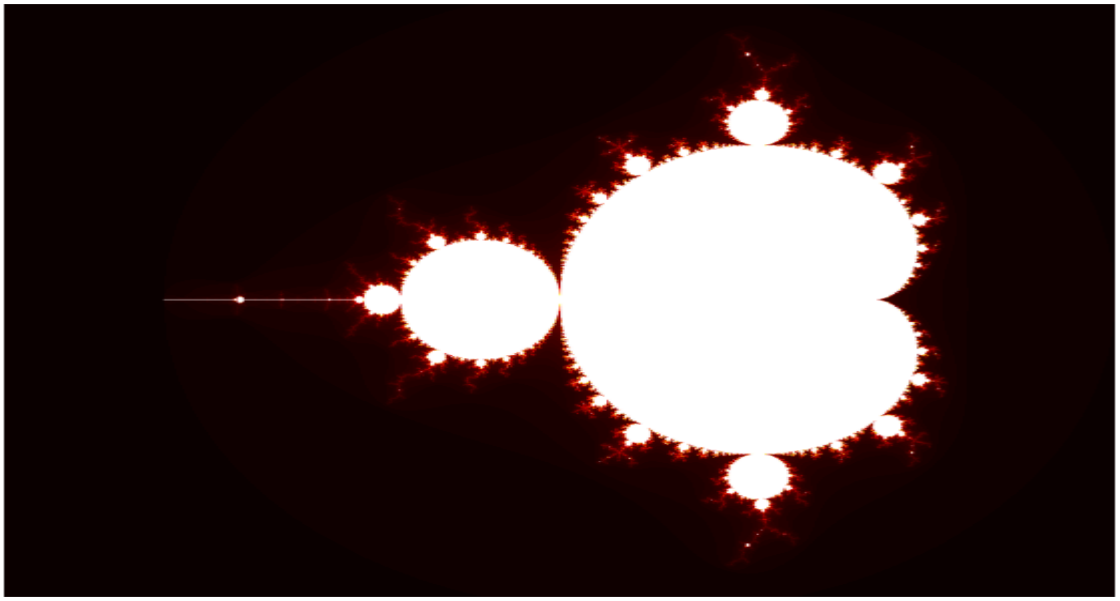
I have used the Dask local version code to implement docstrings and type hints. After this implementation, my code has passed all the ruff checks. Below, I have given a screenshot of a function,

```python
@njit(cache=True)
def mandelbrot_pixel(c_real: float, c_imag: float, max_iter: int) -> int:
    """
    Compute the number of iterations for a single point in the Mandelbrot set.

    Parameters
    ----------
    c_real : float
        Real part of the complex number.
    c_imag : float
        Imaginary part of the complex number.
    max_iter : int
        Maximum number of iterations.

    Returns
    -------
    int
        Number of iterations before divergence, or max_iter if bounded.
    """
    z_real = z_imag = 0.0
    for i in range(max_iter):
        z_sq = z_real * z_real + z_imag * z_imag
        if z_sq > 4.0:
            return i
        z_real, z_imag = (
            z_real * z_real - z_imag * z_imag + c_real,
            2.0 * z_real * z_imag + c_imag,
        )
    return max_iter
```

Screenshot of ruff checks passing:

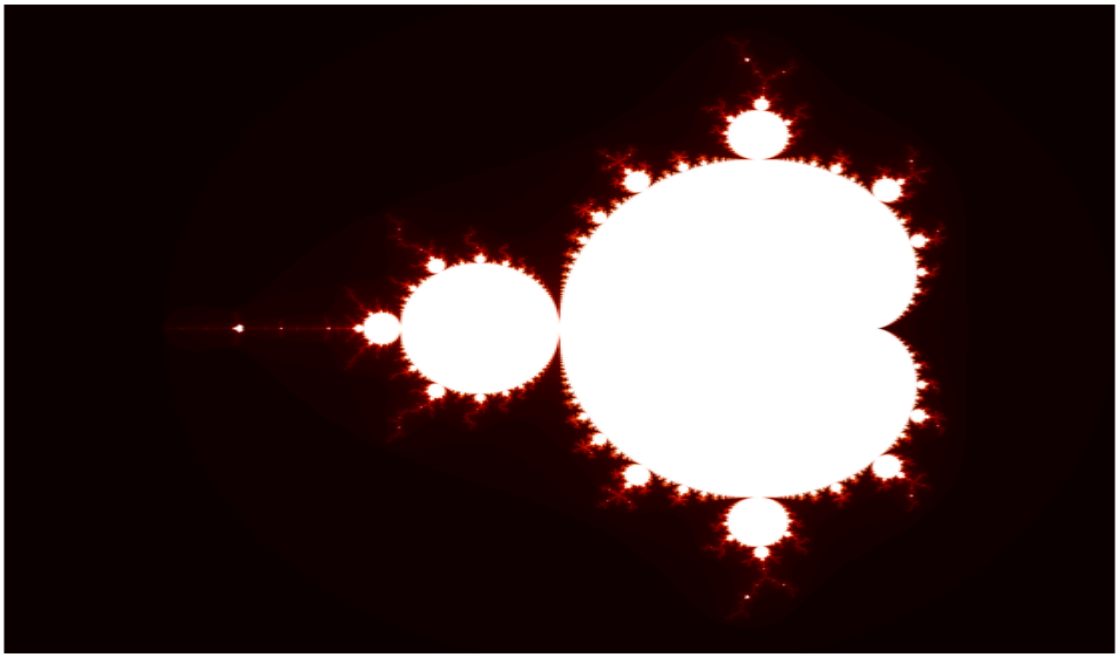
```
(nsc2026) jahid@jahid-ubuntu:~/Aalborg University/second semester/nsc-course/mandelbrot - nsc/
mini_project_2$ ruff check mandelbrot_parallel_chunk_lec6_m2.py
All checks passed!
```

Lecture 10 Milestone 1:



Time taken, GPU 1024x1024: 1.3 ms

Lecture 10 Milestone 2:



Note: My device does not have a dedicated GPU. I have Intel UHD graphics, which do not support float64 computation. So, for the milestone 2 float64 GPU computation, I have used my CPU, and it worked well.

Time taken for f64, CPU 4096x4096: 243.2 ms

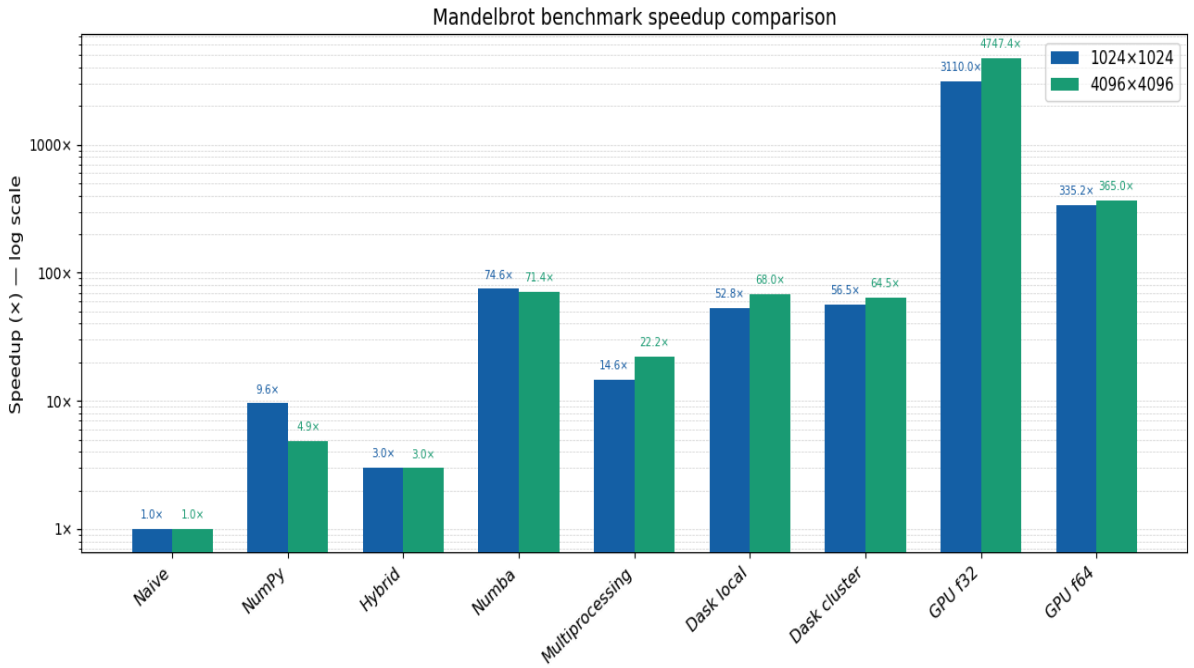
Device	Resolution	Precision	Time (ms)
CPU	4096	64-bit (f64)	243.2 ms
GPU	4096	32-bit (f32)	18.7 ms

Device	Resolution	Precision	Time (ms)
CPU	1024	64-bit (f64)	16.7 ms
GPU	1024	32-bit (f32)	1.8 ms

Lecture 10 Milestone 3: Final benchmark

Implementation	1024×1024 Time (s)	Speedup	4096×4096 Time (s)	Speedup
Naive Python	5.598	1×	88.776	1×
NumPy	0.586	9.55×	18.237	4.87×
Hybrid	1.858	3.01×	29.740	2.99×
Numba (@njit)	0.075	74.26×	1.243	71.44×
Multiprocessing	0.383	14.62×	3.997	22.21×
Dask (local)	0.106	52.58×	1.305	68.00×
Dask (cluster)	0.099	56.82×	1.377	64.48×
GPU (f32)	0.0018	~3100×	0.0187	~4749×
GPU (f64)	0.0167	~335×	0.2432	~365×

Bar chart:


Mandelbrot benchmark speedup comparison
Speedup (×) — log scale
1×
10×
100×
1000×
1024×1024
4096×4096
1.0×
1.0×
9.6×
4.9×
3.0×
3.0×
74.6×
71.4×
14.6×
22.2×
52.8×
68.0×
56.5×
64.5×
3110.0×
4747.4×
335.2×
365.0×
Naive
NumPy
Hybrid
Numba
Multiprocessing
Dask local
Dask cluster
GPU f32
GPU f64